

Final exam in TDT4125 Algorithm construction, advanced course

Exam date	May 19, 2008
Exam time	1500–1900
Grading date	June 9
Language	English
Contact during exam	Magnus Lie Hetland (ph. 91851949)
Allowed tools	All printed and handwritten notes; specific, simple calculator

Please read the entire exam before you start, use your time wisely, and prepare any questions before the lecturer makes his rounds. Make assumptions where needed. Please keep your answers short and concise. Long explanations that do not directly answer a problem are given little or no consideration. Each problem is weighted as shown. For each problem each subproblem is given equal weight.

Problem 1 (20%)

Let the graph G have a structure as shown in Figure 1. The nodes may be joints that are to be inspected by a robot. The edges may be steel or tree beams. All the 27 drawn edges have a length of 1.

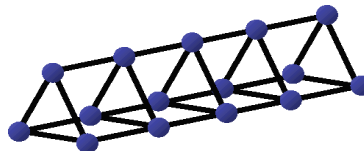


Figure 1

We are to find a shortest possible tour (Traveling Sales Person/TSP tour) through all the $n=15$ nodes, without necessarily following only the drawn (physical) edges. (Assume that the distances between the nodes are measured in straight lines.)

- a. What is the length of the shortest tour in the graph G ? Prove that your answer is correct.
- b. What is the shortest tour that APPROX-TSP-TOUR could find in G ? Also show that this algorithm may find a tour with length $11 + 4\sqrt{2} \approx 16.7$.
- c. What is the shortest tour that Christofides's algorithm could find in G ? Also show that this algorithm may find a tour with length $14 + \sqrt{17} \approx 18.1$.

d. Construct an example, parametrized by n (and only n), which may be related to Figure 1, where Christofides's algorithm may find a tour that asymptotically is 50% longer than the shortest (optimal) tour.

Problem 2 (20%)

Assume that you have two distance functions d and f , where $f(x, y) \geq d(x, y)$ for all x, y . Assume that d is cheaper to compute than f .

a. How can d be used to reduce the cost of a range query with f ?

Assume the situation is reversed: That f is cheaper to compute than d .

b. How can f be used to reduce the cost of a range query with d ?

Assume that you choose the search radius so that only a small minority of the objects in the data set fall inside. Also assume that d and f approximate each other well.

c. Do you think the situation in **a** or **b** will lead to the greatest savings? Explain your reasoning.

d. Do the answers in **a** and **b** hold also for k nearest neighbor searches? Explain your reasoning.

It is generally accepted that it is easier to index distances if the distance distribution has a high standard deviation.

e. Give an example of an application where you think this standard deviation will be high. Explain your reasoning.

Problem 3 (15%)

You have a set of n jobs to do. Each job i has a release time, $r(i)$, and a deadline, $d(i)$. Each job takes 1 hour to perform. A job may not be begun before its release time, and it must be finished before its deadline. Two jobs may not overlap.

a. Briefly describe an algorithm that finds a legal schedule for the jobs, if possible. Show that the algorithm is correct, and give its running time in Θ notation.

Problem 4 (10%)

a. Describe briefly how crossover works in genetic programming, and what its purpose is.

b. Give an example of crossover (before and after) for two trees that represent regular expressions.

Problem 5 (10%)

a. Construct and draw a common suffix tree for the two strings "genetisk" and "nett".

b. Show how you can use this tree to make a substring search for the string "net".

Problem 6 (25%)

Assume that you are given a path that visits all the nodes of a complete, weighted graph G exactly once (a Hamilton path). You are to partition the nodes into two subsets. Let A and B be the paths in G that consist of the nodes in each of these subsets. You wish to minimize $w(A) + w(B)$, that is, the sum of the lengths of the two paths.

- a.** Describe an algorithm that finds such an optimal partitioning. Give its running time in Θ notation.

Assume that there is an algorithm that, in polynomial time, can decide whether a given string belongs to the set L . Let L^* be the set of strings that are concatenation of strings from L .

- b.** Show that there exists an algorithm that, in polynomial time, can decide whether a given string belongs to the set L^* .